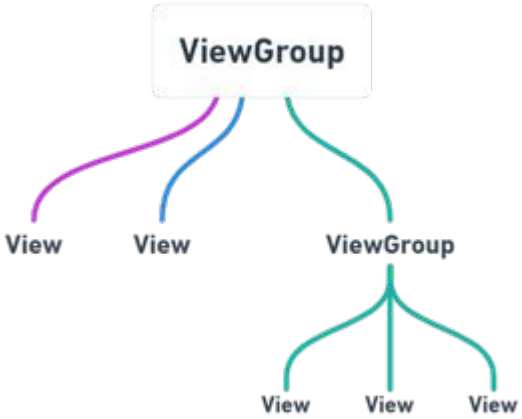
UI Elements in Android

UI Layouts

View: The basic building block for user interface is a View object which is created from the View class and occupies a rectangular area on the screen and is responsible for drawing and event handling. View is the base class for widgets, which are used to create interactive UI components like buttons, text fields, etc.



ViewGroup: The ViewGroup is a subclass of View and provides invisible container that hold other Views or other ViewGroups and define their layout properties.

Types of Android Layout

- **Android Linear Layout:** LinearLayout is a ViewGroup subclass, used to provide child View elements one by one either in a particular direction either horizontally or vertically based on the orientation property.
- **Android Relative Layout:** RelativeLayout is a ViewGroup subclass, used to specify the position of child View elements relative to each other like (A to the right of B) or relative to the parent (fix to the top of the parent).
- **Android Constraint Layout:** ConstraintLayout is a ViewGroup subclass, used to specify the position of layout constraints for every child View relative to other views present. A ConstraintLayout is similar to a RelativeLayout, but having more power.

- **Android Frame Layout:** FrameLayout is a ViewGroup subclass, used to specify the position of View elements it contains on the top of each other to display only a single View inside the FrameLayout.
- **Android Table Layout:** TableLayout is a ViewGroup subclass, used to display the child View elements in rows and columns.
- **Android Web View:** WebView is a browser that is used to display the web pages in our activity layout.
- **Android ListView:** ListView is a ViewGroup, used to display scrollable lists of items in a single column.
- **Android Grid View:** GridView is a ViewGroup that is used to display a scrollable list of items in a grid view of rows and columns.

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http:// schemas.android.com/apk/res/android"
    xmlns:tools="http:// schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <!--EditText with id editText-->
    <EditText
        android:id="@+id/editText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="16dp"
        android:hint="Input"
        android:inputType="text"/>

    <!--Button with id showInput-->
    <Button
        android:id="@+id/showInput"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:text="show"
        android:backgroundTint="@color/colorPrimary"
        android:textColor="@android:color/white"/>

</LinearLayout>
```

Load XML Layout File and its elements from an Activity

```
override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        // finding the button
        val showButton = findViewById<Button>(R.id.showInput)
        // finding the edit text
            val editText = findViewById<EditText>(R.id.editText)
```

Different Attribute of the Layouts

android:id :- This is the ID which uniquely identifies the view.

android:layout_width:- This is the width of the layout.

android:layout_height:- This is the height of the layout

android:layout_marginTop:- This is the extra space on the top side of the layout.

android:layout_marginBottom:- This is the extra space on the bottom side of the layout.

android:layout_marginLeft:- This is the extra space on the left side of the layout.

android:layout_marginRight:- This is the extra space on the right side of the layout.

android:layout_gravity: This specifies how child Views are positioned.

android:layout_weight:- This specifies how much of the extra space in the layout should be allocated to the View.

android:layout_x:- This specifies the x-coordinate of the layout.

android:layout_y:- This specifies the y-coordinate of the layout.

android:layout_width:- This is the width of the layout.

android:paddingLeft:- This is the left padding filled for the layout.

android:paddingRight:- This is the right padding filled for the layout.

android:paddingTop:- This is the top padding filled for the layout.

android:paddingBottom:- This is the bottom padding filled for the layout.

Dp:- Density-independent Pixels

SP:- Scale-independent Pixels

android:layout_width=wrap_content tells your view to size itself to the dimensions required by its content.

android:layout_width=fill_parent tells your view to become as big as its parent view.

Linear Layout

LinearLayout is the most basic layout in android studio, that aligns all the children sequentially either in a horizontal manner or a vertical manner by specifying the **android:orientation** attribute. If one applies **android:orientation=”vertical”** then elements will be arranged one after another in a vertical manner and If you apply **android:orientation=”horizontal”** then elements will be arranged one after another in a horizontal manner.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
        xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:app="http://schemas.android.com/apk/res-auto"
        xmlns:tools="http://schemas.android.com/tools"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        tools:context=".MainActivity">
</LinearLayout>
```

Attributes of Linear Layout

android:layout_weight It is defined individually to the child's views to specify how LinearLayout divides the remaining space amongst the views it contains

android:weightSum Defines the total weight sum
android:orientation How the elements should be arranged in the layout. It can be horizontal or vertical.
android:gravity It specifies how an object should position its content on its X and Y axes.
Possible values are – center_vertical, fill, center, bottom, end, etc.

android:layout_gravity Sets the gravity of the View or Layout relative to its parent.
Possible values are – center_vertical, fill, center, bottom, end, etc.

android:baselineAligned This must be a boolean value, either "true" or "false" and prevents the layout from aligning its children's baselines.

android:id This gives a unique id to the layout.

Relative Layout

Android **RelativeLayout** is a ViewGroup subclass, used to specify the position of child View elements relative to each other like (A to the right of B) or relative to the parent (fix to the top of the parent).

Instead of using LinearLayout, we have to use RelativeLayout to design the user interface and keep our hierarchy flat because it improves the performance of the application.

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:paddingLeft="10dp"
        android:paddingRight="10dp">

        // Add other view or ViewGroup here
</RelativeLayout>
```

Attributes of Relative Layout

layout_alignParentLeft It is set “true” to match the left edge of view to the left edge of parent.

layout_alignParentRight It is set “true” to match the right edge of view to the right edge of parent.

layout_alignParentTop It is set “true” to match the top edge of view to the top edge of parent.

layout_alignParentBottom It is set “true” to match the bottom edge of view to the bottom edge of parent.

layout_alignLeft It accepts another sibling view id and align the view to the left of the specified view id

layout_alignRight It accepts another sibling view id and align the view to the right of the specified view id.

layout_alignStart It accepts another sibling view id and align the view to start of the specified view id.

layout_alignEnd It accepts another sibling view id and align the view to end of specified view id.

layout_centerInParent When it is set “true”, the view will be aligned to the center of parent.

layout_centerHorizontal When it is set “true”, the view will be horizontally centre aligned within its parent.

layout_centerVertical When it is set “true”, the view will be vertically centre aligned within its parent.

layout_toLeftOf It accepts another sibling view id and places the view left of the specified view id.

layout_toRightOf :- It accepts another sibling view id and places the view right of the specified view id.

layout_toStartOf :- It accepts another sibling view id and places the view to start of the specified view id.

layout_toEndOf It accepts another sibling view id and places the view to end of the specified view id.

layout_above It accepts another sibling view id and places the view above the specified view id.

layout_below It accepts another sibling view id and places the view below the specified view id.

ConstraintLayout

ConstraintLayout is similar to that of other View Groups which we have seen in Android such as RelativeLayout, LinearLayout, and many more. In this article, we will take a look at using ConstraintLayout in Android.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
        xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:app="http://schemas.android.com/apk/res-auto"
        xmlns:tools="http://schemas.android.com/tools"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:context=".MyActivity">
```

```
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="16dp"
        android:layout_marginRight="16dp"
        android:gravity="center"
        android:padding="10dp"
        android:text="Hello World"
        android:textColor="@color/black"
        android:textSize="20sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Attributes of Constraint Layout

android:id This is used to give a unique id to the layout.
app:layout_constraintBottom_toBottomOf This is used to constrain the view with respect to the bottom position.
app:layout_constraintLeft_toLeftOf This attribute is used to constrain the view with respect to the left position.
app:layout_constraintRight_toRightOf This attribute is used to constrain the view with respect to the right position.
app:layout_constraintTop_toTopOf This attribute is used to constrain the view with respect to the top position.

Advantages of using ConstraintLayout in Android

- ConstraintLayout provides you the ability to completely design your UI with the drag and drop feature provided by the Android Studio design editor.
- It helps to improve the UI performance over other layouts.
- With the help of ConstraintLayout, we can control the group of widgets through a single line of code.
- With the help of ConstraintLayout, we can easily add animations to the UI components which we used in our app.

Disadvantages of Constraint Layout

- When we use the Constraint Layout in our app, the XML code generated becomes a bit difficult to understand.
- In most of the cases, the result obtain will not be the same as we got to see in the design editor.
- Sometimes we have to create a separate layout file for handling the UI for the landscape mode.

FrameLayout

Frame Layout is designed to block out an area on the screen to display a single item. Generally, FrameLayout should be used to hold a single child view, because it can be difficult to organize child views in a way that's scalable to different screen sizes without the children overlapping each other.

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    // Add items or widgets here
    </FrameLayout>
```

Attributes of FrameLayout

android:id This is the ID which uniquely identifies the layout.

android:foreground This defines the drawable to draw over the content and possible values may be a color value, in the form of "#rgb", "#argb", "#rrggbb", or "#aarrggbb".

android:foregroundGravity Defines the gravity to apply to the foreground drawable. The gravity defaults to fill. Possible values are top, bottom, left, right, center, center_vertical, center_horizontal etc.

android:measureAllChildren Determines whether to measure all children or just those in the VISIBLE or INVISIBLE state when measuring. Defaults to false.

TableLayout

Android **TableLayout** is a ViewGroup subclass which is used to display the child View elements in rows and columns. It will arrange all the children elements into rows and columns and does not display any border lines in between rows, columns or cells.

The working of TableLayout is almost similar to HTML table and it contains as many columns as row with the most cells.

```
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="10dp"
        android:paddingLeft="5dp"
        android:paddingRight="5dp">
        // Add Table rows here
</TableLayout>
```

To Declare Rows

```
<TableRow android:background="#51B435" android:padding="10dp">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Rank" />
</TableRow>
```

Attributes of TableLayout

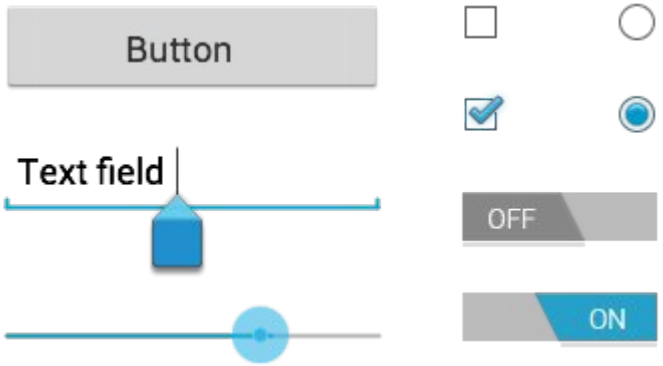
android:id This is the ID which uniquely identifies the layout.

android:collapseColumns This specifies the zero-based index of the columns to collapse. The column indices must be separated by a comma: 1, 2, 5.

android:shrinkColumns The zero-based index of the columns to shrink. The column indices must be separated by a comma: 1, 2, 5.

android:stretchColumns The zero-based index of the columns to stretch. The column indices must be separated by a comma: 1, 2, 5.

UI Controls



UI Elements

A View is an object that draws something on the screen that the user can interact with and a ViewGroup is an object that holds other View (and ViewGroup) objects in order to define the layout of the user interface.

You define your layout in an XML file which offers a human-readable structure for the layout, similar to HTML. For example, a simple vertical layout with a text view and a button looks like this −

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="fill_parent"
   android:layout_height="fill_parent"
   android:orientation="vertical" >

      <TextView android:id="@+id/text"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="I am a TextView" />

   <Button android:id="@+id/button"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="I am a Button" />
</LinearLayout>
```

Android UI Controls

TextView This control is used to display text to the user.

EditText EditText is a predefined subclass of TextView that includes rich editing capabilities.

AutoCompleteTextView The AutoCompleteTextView is a view that is similar to EditText, except that it shows a list of completion suggestions automatically while the user is typing.

Button A push-button that can be pressed, or clicked, by the user to perform an action.

ImageButton An ImageButton is an AbsoluteLayout which enables you to specify the exact location of its children. This shows a button with an image (instead of text) that can be pressed or clicked by the user.

CheckBox An on/off switch that can be toggled by the user. You should use check box when presenting users with a group of selectable options that are not mutually exclusive.

ToggleButton An on/off button with a light indicator.

RadioButton The RadioButton has two states: either checked or unchecked.

RadioGroup A RadioGroup is used to group together one or more RadioButtons.

ProgressBar The ProgressBar view provides visual feedback about some ongoing tasks, such as when you are performing a task in the background.

Spinner A drop-down list that allows users to select one value from a set.

TimePicker The TimePicker view enables users to select a time of the day, in either 24-hour mode or AM/PM mode.

DatePicker The DatePicker view enables users to select a date of the day.

Creating UI Controls

A view object may have a unique ID assigned to it which will identify the View uniquely within the tree. The syntax for an ID, inside an XML tag is

android:id="@+id/text_id"

To create a UI Control/View/Widget you will have to define a view/widget in the layout file and assign it a unique ID as follows −

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="fill_parent"
   android:layout_height="fill_parent"
   android:orientation="vertical" >

   <TextView android:id="@+id/text_id"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="I am a TextView" />
</LinearLayout>
```

Then finally create an instance of the Control object and capture it from the layout, use the following −

```
TextView myText = (TextView) findViewById(R.id.text_id);
```

Event Handling

Events are a useful way to collect data about a user's interaction with interactive components of Applications. Like button presses or screen touch etc. The Android framework maintains an event queue as first-in, first-out (FIFO) basis. You can capture these events in your program and take appropriate action as per requirements.

There are following three concepts related to Android Event Management −

Event Listeners − An event listener is an interface in the View class that contains a single callback method. These methods will be called by the Android framework when the View to which the listener has been registered is triggered by user interaction with the item in the UI.

Event Listeners Registration − Event Registration is the process by which an Event Handler gets registered with an Event Listener so that the handler is called when the Event Listener fires the event.

Event Handlers − When an event happens and we have registered an event listener for the event, the event listener calls the Event Handlers, which is the method that actually handles the event.

Event Listeners & Event Handlers

OnClickListener()

This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. You will use onClick() event handler to handle such event.

OnLongClickListener()

This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. for one or more seconds. You will use onLongClick() event handler to handle such event.

OnFocusChangeListener()

This is called when the widget looses its focus ie. user goes away from the view item. You will use onFocusChange() event handler to handle such event.

OnTouchListener()

This is called when the user presses the key, releases the key, or any movement gesture on the screen. You will use onTouch() event handler to handle such event.

OnMenuItemClickListener()

This is called when the user selects a menu item. You will use

onCreateContextMenuItemListener()

This is called when the context menu is being built(as the result of a sustained "long click)

Event Listeners Registration

Touch Mode

Users can interact with their devices by using hardware keys or buttons or touching the screen.Touching the screen puts the device into touch mode. The user can then interact with it by touching the on-screen virtual buttons, images, etc.You can check if the device is in touch mode by calling the View class's isInTouchMode() method.

Focus

A view or widget is usually highlighted or displays a flashing cursor when it's in focus. This indicates that it's ready to accept input from the user.

isFocusable() − it returns true or false

isFocusableInTouchMode() − checks to see if the view is focusable in touch mode. (A view may be focusable when using a hardware key but not when the device is in touch mode)

```
public boolean onTouchEvent(motionEvent event){
   switch(event.getAction()){
      case TOUCH_DOWN:
      Toast.makeText(this,"you have clicked down Touch button",Toast.LENTH_LONG).show();
      break();
      case TOUCH_UP:
      Toast.makeText(this,"you have clicked up touch button",Toast.LENTH_LONG).show();
      break;
      case TOUCH_MOVE:
      Toast.makeText(this,"you have clicked move touch button"Toast.LENTH_LONG).show();
      break;
   }
   return super.onTouchEvent(event) ;
}
```

TextView

Widget refers to the elements of the UI (User Interface) that helps user interacts with the Android App. **TextView** is one of many such widgets which can be used to improve the UI of the app.

TextView refers to the widget which displays some text on the screen based on the layout, size, colour, etc set for that particular TextView. It optionally allows us to modify or edit itself as well.

Class Syntax:

```
public class TextView

  extends View

    implements ViewTreeObserver.OnPreDrawListener
```

Example

```
<LinearLayout
      xmlns:android="http://schemas.android.com/apk/res/android"
      android:layout_width="match_parent"
      android:layout_height="match_parent">

    <TextView
        android:id="@+id/text_view_id"
        android:layout_height="wrap_content"
        android:layout_width="wrap_content"
        android:text="Hello World" />

     </LinearLayout>
```

Styles and Themes

A style resource defines the format and look for a UI. A style can be applied to an individual View (from within a layout file) or to an entire Activity or application (from within the manifest file).

```
<?xml version="1.0" encoding="utf-8"?>

<resources>

  <style name="CustomFontStyle">

    <item name="android:layout_width">matchl_parent</item>

    <item name="android:layout_height">wrap_content</item>

    <item name="android:capitalize">characters</item>

    <item name="android:typeface">monospace</item>

    <item name="android:textSize">12sp</item>

    <item name="android:textColor">#00FF00</item>/>

  </style>

</resources>
```

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"

  android:layout_width="fill_parent"

  android:layout_height="fill_parent"

  android:orientation="vertical" >

  <TextView

    android:id="@+id/text_id"

    style="@style/CustomFontStyle"

    android:text="@string/hello_world" />

</LinearLayout>
```

To set a theme for all the activities of your application, open the AndroidManifest.xml file

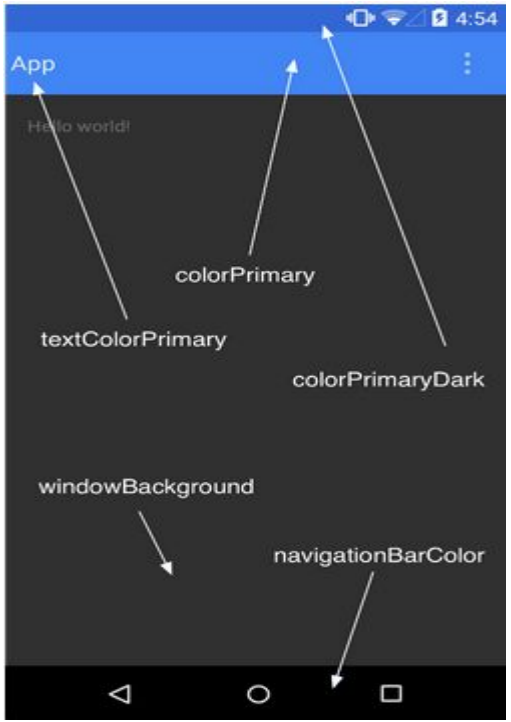
```
  <application android:theme="@style/CustomFontStyle">
```

But if you want a theme applied to just one Activity in your application, then add the android:theme attribute to the <activity> tag only.

```
  <activity android:theme="@style/CustomFontStyle">
```

Themes

```
<resource>
  <style name="AppTheme" parent="android:Theme.Material">
    <item name ="android:color/primary">@color/primary</item>
    <item name ="android:color/primaryDark">@color/primary_dark</item>
    <item name ="android:colorAccent/primary">@color/accent</item>
  </style>
<resource>
```



Toast

A toast provides a simple popup message that is displayed on the current activity UI screen

```
// to get Context

Context context = getApplicationContext();

// message to display

String text = "Toast message";

// toast time duration, can also set manual value

int duration = Toast.LENGTH_SHORT;

Toast toast = Toast.makeText(context, text, duration);

// to show the toast

toast.show();
```

Toggle Button

Android Toggle Button can be used to display checked/unchecked (On/Off) state on the button.

It is beneficial if user have to change the setting between two states. It can be used to On/Off Sound, Wifi, Bluetooth etc.

Since Android 4.0, there is another type of toggle button called *switch* that provides slider control.

Attributes

android:disabledAlpha: The alpha to apply to the indicator when disabled.

android:textOff: The text for the button when it is not checked.

android:textOn: The text for the button when it is checked.

Methods of toggle button

CharSequence getTextOff(): Returns the text when button is not in the checked state.
CharSequence getTextOn(): Returns the text for when button is in the checked state.
void setChecked(boolean checked): Changes the checked state of this button.

```
<ToggleButton
    android:id="@+id/toggleButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerHorizontal="true"
    android:layout_centerVertical="true"
    android:onClick="onToggleClick"   />
```

```
togglebutton
        = (ToggleButton)findViewById(R.id.toggleButton);
```

```
public void onToggleClick(View view)
    {
        if (togglebutton.isChecked()) {
            textview.setText("Toggle is ON");
        }
        else {
            textview.setText("Toggle is OFF");
        }
    }
```

CheckBox

Android CheckBox is a type of two state button either checked or unchecked.

There can be a lot of usage of checkboxes. For example, it can be used to know the hobby of the user, activate/deactivate the specific action etc.

Android CheckBox class is the subclass of CompoundButton class.

Methods of checkbox

public boolean isChecked(): Returns true if it is checked otherwise false.

public void setChecked(boolean status): Changes the state of the CheckBox.

RadioButton

RadioButton is a two states button which is either checked or unchecked. If a single radio button is unchecked, we can click it to make checked radio button. Once a radio button is checked, it cannot be marked as unchecked by user.

RadioButton is generally used with *RadioGroup*. RadioGroup contains several radio buttons, marking one radio button as checked makes all other radio buttons as unchecked.

```
<RadioGroup
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/radioGroup">

    <RadioButton
        android:id="@+id/radioMale"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="  Male"
        android:layout_marginTop="10dp"
        android:checked="false"
        android:textSize="20dp" />

    <RadioButton
        android:id="@+id/radioFemale"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="   Female"
        android:layout_marginTop="20dp"
        android:checked="false"
        android:textSize="20dp" />
</RadioGroup>
```

```
public class MainActivity extends AppCompatActivity {
    Button button;
    RadioButton genderradioButton;
    RadioGroup radioGroup;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        radioGroup=(RadioGroup)findViewById(R.id.radioGroup);
    }
    public void onclickbuttonMethod(View v){
        int selectedId = radioGroup.getCheckedRadioButtonId();
        genderradioButton = (RadioButton) findViewById(selectedId);
        if(selectedId==-1){
            Toast.makeText(MainActivity.this,"Nothing selected", Toast.LENGTH_SHORT).show();
        }
        else{
            Toast.makeText(MainActivity.this,genderradioButton.getText(), Toast.LENGTH_SHORT).show();
        }

    }
```

AlertDialog

Android AlertDialog can be used to display the dialog message with OK and Cancel buttons. It can be used to interrupt and ask the user about his/her choice to continue or discontinue.

Android AlertDialog is composed of three regions: title, content area and action buttons.

Android AlertDialog is the subclass of Dialog class.

Methods of AlertDialog

public AlertDialog.Builder setTitle(CharSequence): This method is used to set the title of AlertDialog.

public AlertDialog.Builder setMessage(CharSequence): This method is used to set the message for AlertDialog.

public AlertDialog.Builder setIcon(int): This method is used to set the icon over AlertDialog.

```
AlertDialog.Builder builder = new AlertDialog.Builder(MainActivity.this);
        // Set the message show for the Alert time
       builder.setMessage("Do you want to exit ?");
        // Set Alert Title
       builder.setTitle("Alert !");
        // Set Cancelable false for when the user clicks on the outside the Dialog Box then it will remain show
       builder.setCancelable(false);
        // Set the positive button with yes name Lambda OnClickListener method is use of DialogInterface interface.
       builder.setPositiveButton("Yes", (DialogInterface.OnClickListener) (dialog, which) -> {
           // When the user click yes button then app will close
           finish();
       });
        // Set the Negative button with No name Lambda OnClickListener method is use of DialogInterface interface.
       builder.setNegativeButton("No", (DialogInterface.OnClickListener) (dialog, which) -> {
           // If user click no then dialog box is canceled.
           dialog.cancel();
       });

       // Create the Alert dialog
       AlertDialog alertDialog = builder.create();
       // Show the Alert Dialog box
       alertDialog.show();
```

Spinner

Android Spinner is like the combox box of AWT or Swing. It can be used to display the multiple options to the user in which only one item can be selected by the user.

Android spinner is like the drop down menu with multiple values from which the end user can select only one value.

Android spinner is associated with AdapterView. So you need to use one of the adapter classes with spinner.

Android:id: Used to specify the id of the view.
android:textAlignment: Used to the text alignment in the dropdown list.
android:background: Used to set the background of the view.
android:padding: Used to set the padding of the view.
android:visibility: Used to set the visibility of the view.
android:gravity: Used to specify the gravity of the view like center, top, bottom, etc

```
<Spinner
        android:id="@+id/coursesspinner"
        android:layout_height ="50dp"
        android:layout_width ="160dp"
        android:layout_marginEnd ="10dp"
        android:layout_marginStart ="10dp"
        android:layout_marginBottom ="10dp"
        android:layout_marginTop ="10dp"
        app:layout_constraintStart_toStartOf ="parent"
        app:layout_constraintTop_toTopOf ="parent"/>
```

```
  String[] courses = { "C", "Data structures","Interview prep", "Algorithms",
"DSA with java", "OS" };
     @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
         // Take the instance of Spinner and
        // apply OnItemSelectedListener on it which
        // tells which item of spinner is clicked
        Spinner spino = findViewById(R.id.coursesspinner);
        spin.setOnItemSelectedListener(this);
         // Create the instance of ArrayAdapter
        // having the list of courses
     ArrayAdapter ad = new
     ArrayAdapter(this,android.R.layout.simple_spinner_item,courses);
         ad.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
        // Set the ArrayAdapter (ad) data on the
        // Spinner which binds data to spinner
        spino.setAdapter(ad);
    }
```

Autocomplete TextView

In this example, we are displaying the programming languages in the autocompletetextview. All the programming languages are stored in string array. We are using the **ArrayAdapter** class to display the array content.

```
<AutoCompleteTextView
        android:layout_width="match_parent"
        android:layout_height="100px"
        android:placeholder="Enter your country name"
        android:id="@+id/txtcountries"/>
```

```
public class MainActivity extends Activity {
    String[] countries={"India","Australia","West indies","indonesia","Indiana",
                        "South Africa","England","Bangladesh","Srilanka","singapore"};
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        ArrayAdapter<String> adapter=new
ArrayAdapter<String>(this,android.R.layout.simple_dropdown_item_1line,countries);
AutoCompleteTextView textView=(AutoCompleteTextView)findViewById(R.id.txtcountries);
        textView.setThreshold(3);
        textView.setAdapter(adapter);
    }
}
```

RatingBar

- **RatingBar** is a UI widget that allows users to give **star ratings** ⭐ (like feedback in apps).
- It is based on **SeekBar** but shows stars instead of a sliding bar.

Property	Description
android:numStars:	Total number of stars to display.
android:stepSize:	Increment value (e.g., 1.0 for whole stars, 0.5 for half stars).
Android:rating:	Default rating (how many stars are filled initially).
android:isIndicator:	If true, user cannot change rating (read-only).

`getRating()` Returns the current rating (float).

setRating(float rating) Sets the rating programmatically.

setStepSize(float stepSize) Sets how much the rating increases/decreases per step.

getNumStars() Gets total number of stars.

```
<!-- res/layout/activity_main.xml -->
<RatingBar
    android:id="@+id/ratingBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:numStars="5"
    android:stepSize="1.0"
    android:rating="2"/>
```

```
// MainActivity.java
RatingBar ratingBar = findViewById(R.id.ratingBar);
Button btnSubmit = findViewById(R.id.btnSubmit);

btnSubmit.setOnClickListener(v -> {
    float rating = ratingBar.getRating();
    Toast.makeText(this, "You rated: " + rating + " stars", Toast.LENGTH_SHORT).show();
});
```

WebView

WebView is a view that displays web pages inside the application. It is used to turn the application into a web application.

```
// Find the WebView by its unique ID
    WebView webView = findViewById(R.id.web);

    webView.loadUrl("https://www.google.com");

    // this will enable the javascript.
    webView.getSettings().setJavaScriptEnabled(true);

    // WebViewClient allows you to handle
    // onPageFinished and override Url loading.
    webView.setWebViewClient(new WebViewClient());
```

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:tools="http://schemas.android.com/tools"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:context="com.gfg.android.examplekotlin.MainActivity">

        <WebView
                android:id="@+id/web"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                tools:layout_editor_absoluteX="8dp"
                tools:layout_editor_absoluteY="8dp" />
</RelativeLayout>
```

In **AndroidManifest.xml**, one needs to include the below permission, in order to access the internet

```
<uses-permission android:name="android.permission.INTERNET" />
```

SeekBar

Android SeekBar is a kind of ProgressBar with draggable thumb. The end user can drag the thum left and right to move the progress of song, file download etc.

The SeekBar.OnSeekBarChangeListener interface provides methods to perform even handling for seek bar.

Android SeekBar and RatingBar classes are the sub classes of AbsSeekBar.

setOnSeekBarChangeListener interface is used which provides three methods.

1. **onProgressChanged**: In this method progress is changed and then according to this change the progress value can used in our logic.
2. **onStartTrackingTouch**: In this method when the user has started dragging, then this method will be called automatically.
3. **onStopTrackingTouch**: In this method, when the user stops dragging, then this method will called automatically.

```
Text_message = (TextView)findViewById(R.id.message_id);
seekbar = (SeekBar)findViewById(R.id.seekbar);
seekbar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
@Override public void onProgressChanged(SeekBar seekBar,int progress,boolean fromUser){
                        message.setTextSize(progress + 1);
 }
@Override public void onStartTrackingTouch(SeekBar seekBar){}
@Override public void onStopTrackingTouch(SeekBar seekBar){}Progress });
        }
```

DatePicker Dialog

DatePicker dialog is seen used in many android applications where we have to select the date.

This widget is mostly seen in the hotel reservation applications for travel booking applications.

With the help of this widget, we can simply pick the date from the DatePicker dialog.

```
public String getCurrentDate(){
        StringBuilder builder=new StringBuilder();;
        builder.append((picker.getMonth() + 1)+"/");//month is 0 based
        builder.append(picker.getDayOfMonth()+"/");
        builder.append(picker.getYear());
        return builder.toString();
    }
```

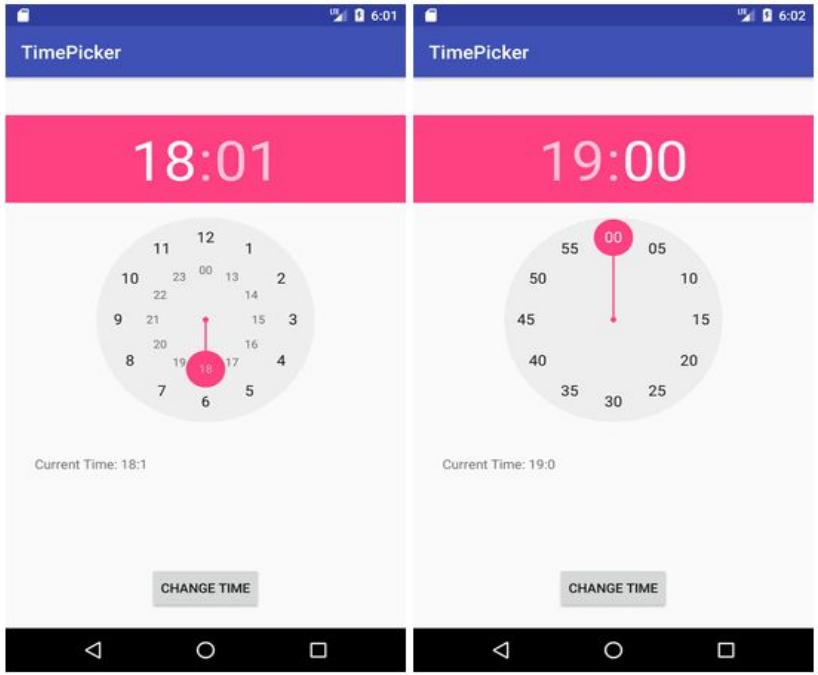
```
<DatePicker
        android:id="@+id/datePicker"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+id/textView1"
        android:layout_centerHorizontal="true"
        android:layout_marginBottom="36dp" />
```

```
picker=(DatePicker)findViewById(R.id.datePicker);
```

TimePicker

Android TimePicker widget is used to select date. It allows you to select time by hour and minute. You cannot select time by seconds.

The android.widget.TimePicker is the subclass of FrameLayout class.



Syntax in xml

```
<TimePicker
        android:id="@+id/timePicker"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+id/textView1"
        android:layout_centerHorizontal="true"
    android:layout_marginBottom="36dp" />
```

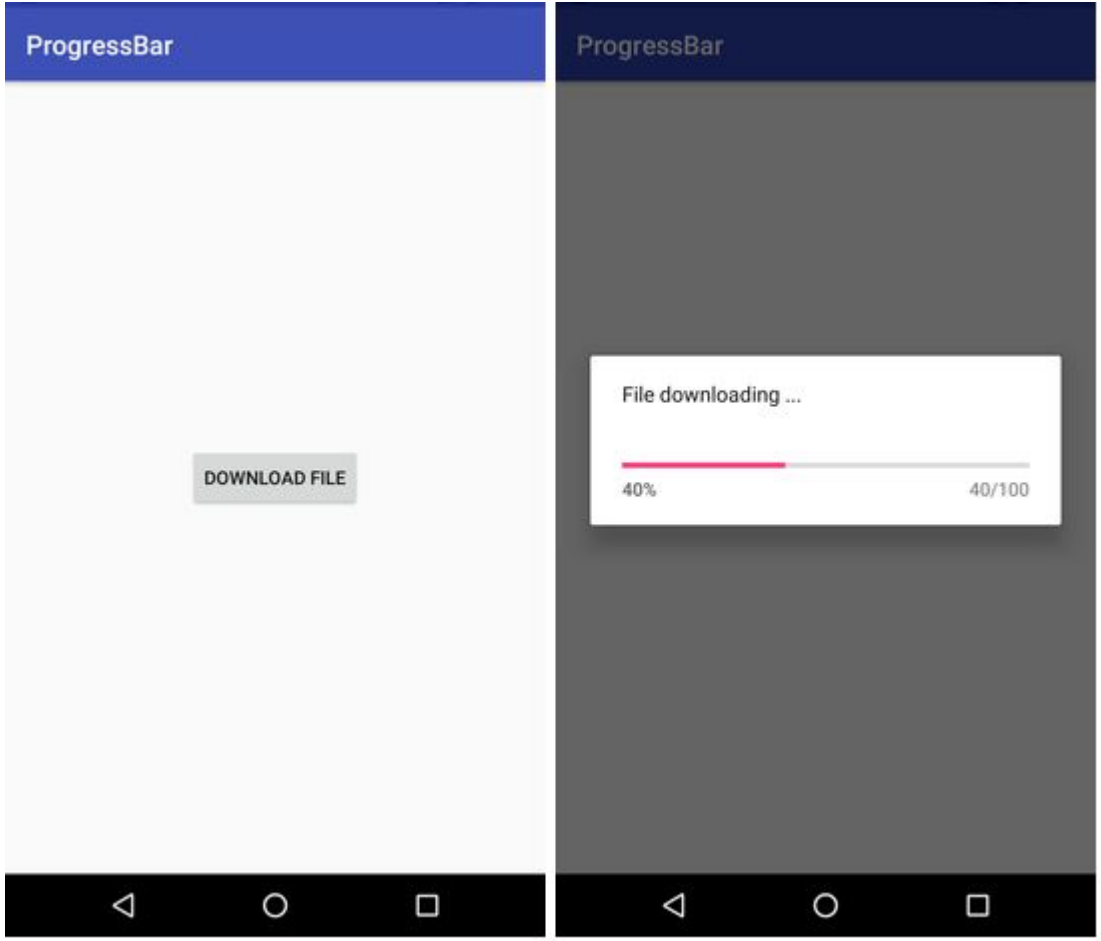
Java

```
    public String getCurrentTime(){
String currentTime="Current Time: timepicker.getCurrentHour()+":"+timepicker.getCurrentMinute();
return currentTime;      }
```

Progressbar

We can display the android progress bar dialog box to display the status of work being done e.g. downloading file, analyzing status of work etc.

```
ProgressDialog progressBar = new ProgressDialog(this);
progressBar.setCancelable(true);//you can cancel it by pressing back button
progressBar.setMessage("File downloading ...");
progressBar.setProgressStyle(ProgressDialog.STYLE_HORIZONTAL);
progressBar.setProgress(0);//initially progress is 0
progressBar.setMax(100);//sets the maximum value 100
progressBar.show();//displays the progress bar
```


ProgressBar
DOWNLOAD FILE
ProgressBar
File downloading ...
40%
40/100

Android Analog clock and Digital clock

The **android.widget.AnalogClock** and **android.widget.DigitalClock** classes provides the functionality to display analog and digital clocks.

Android analog and digital clocks are used to show time in android application.

Android AnalogClock is the subclass of View class.

Android DigitalClock is the subclass of TextView class. Since Android API level 17, it is *deprecated*. You are recommended to use **TextClock** Instead.

The AnalogClock was deprecated in API level 23. This widget is no longer supported. Instead if you want to use AnalogClock in your application you need to hard code. It does not appear in API level 27 to drag from palette.

In android, you need to drag analog and digital clocks from the pallet to display analog and digital clocks.

```
<AnalogClock
    android:id="@+id/analogClock1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentTop="true"
    android:layout_centerHorizontal="true"
    android:layout_marginLeft="136dp"
    android:layout_marginTop="296dp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

<DigitalClock

android:id="@+id/digitalClock1"

android:layout_width="wrap_content"

android:layout_height="wrap_content"

android:layout_below="@+id/analogClock1"

android:layout_centerHorizontal="true"

android:layout_marginLeft="176dp"

android:layout_marginTop="84dp"

android:text="DigitalClock"

app:layout_constraintStart_toStartOf="parent"

1. app:layout_constraintTop_toTopOf="parent" **/>**


10:59
AnalogDigital
10:59 AM

ScrollView

ScrollView class provides the functionality of scroll view.

ScrollView is used to scroll the child elements of palette inside ScrollView.

Android supports vertical scroll view as default scroll view.

Vertical ScrollView scrolls elements vertically.

Android uses *HorizontalScrollView* for horizontal ScrollView.

Switcher

A ViewSwitcher control contains two child View objects and only one of those is shown at a time.

It switches between the two, animating as it does so.

Primarily, the ImageSwitcher and TextSwitcher objects are used.

Each one provides a way to set a new child View, either a Drawable resource or a text string, and then animates from what is displayed to the new contents.

Image Slider

Android *image slider* slides one entire screen to another screen.

Image slider is created by **ViewPager** which is provided by support library.

To implement image slider, you need to inherit ViewPager class which extends PagerAdapter.

```
<android.support.v4.view.ViewPager
    android:id="@+id/viewPage"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent" />
```

```
ViewPager mViewPager = (ViewPager) findViewById(R.id.viewPage);
ImageAdapter adapterView = new ImageAdapter(this);
mViewPager.setAdapter(adapterView);
```

We need to override following methods of PagerAdapter class.

1. **isViewFromObject(View, Object):** This method checks the view whether it is associated with key and returned by instantiateItem().
2. **instantiateItem(ViewGroup, int):** This method creates the page position passed as an argument.
3. **destroyItem(ViewGroup, int, Object):** It removes the page from its current position from container. In this example we simply removed object using removeView().
4. **getCount():** It returns the number of available views in ViewPager.

TabLayout

TabLayout is used to implement horizontal tabs. TabLayout is released by Android after the deprecation of *ActionBar.TabListener (API level 21)*.

TabLayout is introduced in design support library to implement tabs.

Tabs are created using *newTab()* method of TabLayout class. The title and icon of Tabs are set through *setText(int)* and *setIcon(int)* methods of TabListener interface respectively. Tabs of layout are attached over TabLayout using the method addTab(Tab) method.

```
TabLayout tabLayout = (TabLayout)findViewById(R.id.tabLayout);

tabLayout.addTab(tabLayout.newTab().setText("Tab 1"));

tabLayout.addTab(tabLayout.newTab().setText("Tab 2"));

tabLayout.addTab(tabLayout.newTab().setText("Tab 3"));
```

the dependency library of TabLayout in build.gradle file.

“implementation 'com.android.support:design:26.1.0' “

```
<android.support.design.widget.TabLayout
    android:id="@+id/tabLayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="#1db995"/>
  <android.support.v4.view.ViewPager
    android:id="@+id/viewPager"
    android:layout_width="355dp"
    android:layout_height="455dp"
    app:layout_constraintTop_toBottomOf="@+id/tabLayout"
   tools:layout_editor_absoluteX="8dp" />
```

```
tabLayout=(TabLayout)findViewById(R.id.tabLayout);
    viewPager=(ViewPager)findViewById(R.id.viewPager);
    tabLayout.addTab(tabLayout.newTab().setText("Home"));
    tabLayout.addTab(tabLayout.newTab().setText("Sport"));
    tabLayout.addTab(tabLayout.newTab().setText("Movie"));
    tabLayout.setTabGravity(TabLayout.GRAVITY_FILL);
     final MyAdapter adapter = new MyAdapter(this,getSupportFragmentManager(), tabLayout.getTabCount());
    viewPager.setAdapter(adapter);
     viewPager.addOnPageChangeListener(new TabLayout.TabLayoutOnPageChangeListener(tabLayout));
     tabLayout.addOnTabSelectedListener(new TabLayout.OnTabSelectedListener() {
        @Override            public void onTabSelected(TabLayout.Tab tab) {
           viewPager.setCurrentItem(tab.getPosition());            }
        @Override   public void onTabUnselected(TabLayout.Tab tab) {   }
        @Override   public void onTabReselected(TabLayout.Tab tab) {   }
    });
```

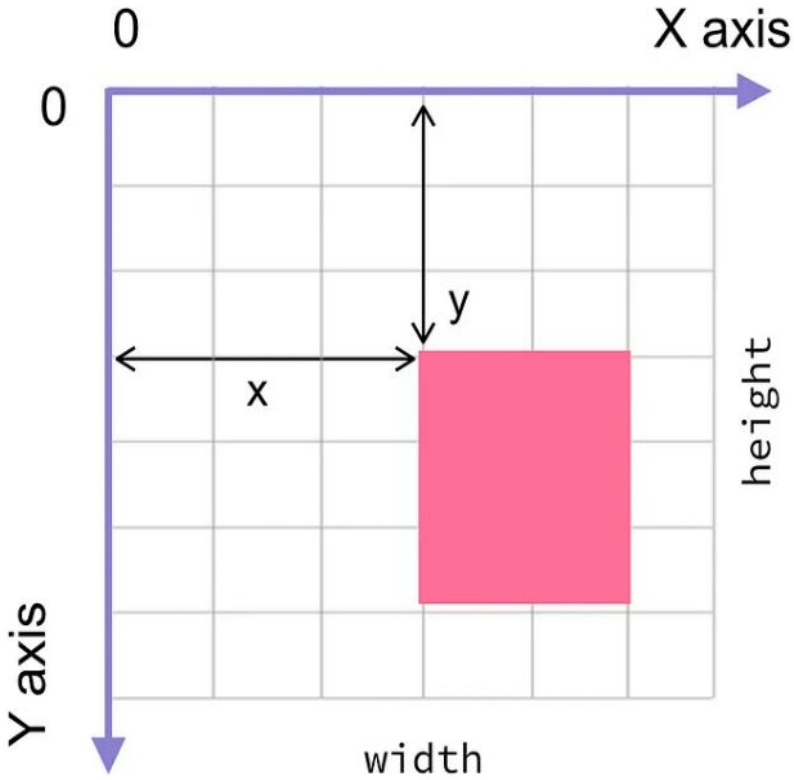
Drawing in android

The **android.graphics.Canvas** can be used to draw graphics in android. It provides methods to draw oval, rectangle, picture, text, line etc.

The **android.graphics.Paint** class is used with canvas to draw objects. It holds the information of color and style.

The coordinate system of the Android canvas starts in the top left corner, where [0,0] represents that point. The y axis is positive downwards, and x axis positive towards the right.

All elements drawn on a canvas are placed relative to the [0,0] point.



How do I use a Canvas?

To draw onto a canvas in Android, you will need four things:

1. A bitmap or a view — to hold the pixels where the canvas will be drawn.
2. Canvas — to run the drawing commands on.
3. Drawing commands — to indicate to the canvas **what** to draw.
4. Paint — to describe **how** to draw the commands.

Get access to a Canvas instance

In order to get access to a **Canvas** instance, you will need to create a class that extends from **View**. This will then allow you to override the **onDraw** method, which has a **Canvas** as a parameter.

```
class CustomView @JvmOverloads constructor(context: Context,
    attrs: AttributeSet? = null, defStyleAttr: Int = 0)
    : View(context, attrs, defStyleAttr) {
    // Called when the view should render its content.
    override fun onDraw(canvas: Canvas?) {
        super.onDraw(canvas)
        // DRAW STUFF HERE   }}
```

Add this in XML

```
<za.co.riggaroo.customviews.CustomView
        android:layout_width="200dp"
        android:layout_height="300dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent"/>
```

You can create canvas programmatically

```
val bitmap = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888)
val canvas = Canvas(bitmap)
```

To draw a circle onto the view, give it a center point x,y , its size and a paint object:

```
canvas.drawCircle(x, y, size, paint)
```

Another one is the drawRect() method. This draws a rectangle on screen:

```
canvas.drawRect(rect, paint)
```

Paint

The Paint class is possibly the most interesting graphics class. There is so much that a Paint object can do that can really make your drawing operations shine.

The Paint class typically holds colour and style information. The Paint object is then used for drawing objects (i.e. bitmap, text, paths etc) onto a Canvas.

To create a Paint object:

```
private val textPaint =   Paint().apply {
        isAntiAlias = true
        color = Color.RED
        style = Paint.Style.STROKE
    }
```

The isAntiAlias flag is quite an important one. If you are drawing objects to your canvas and you notice that the edges of your objects have jagged edges, it is likely that you haven't set this flag to true. This flag indicates to the paint to smooth out the edges of the object you are drawing to the canvas.

The Paint class has more than just those three properties, there are many more things you can do with it. For instance, you can also set properties related to text rendering, such as the typeface, letterSpacing(kerning) and textSize.

```
private val textPaint =   Paint().apply {
      isAntiAlias = true
      textSize = fontSize
      letterSpacing = letterSpace
      typeface = newTypeface
      setShadowLayer(blurValue, x, y, Color.BLACK)
  }
```

Demo code for using canvas and paint

```
// custom drawing code here
        Paint paint = new Paint();
        paint.setStyle(Paint.Style.FILL);
        // make the entire canvas white
        paint.setColor(Color.WHITE);
        canvas.drawPaint(paint);
        // draw blue circle with anti aliasing turned off
        paint.setAntiAlias(false);
        paint.setColor(Color.BLUE);
        canvas.drawCircle(20, 20, 15, paint);
```

```
// draw green circle with anti aliasing turned on
    paint.setAntiAlias(true);
    paint.setColor(Color.GREEN);
    canvas.drawCircle(60, 20, 15, paint);
    // draw red rectangle with anti aliasing turned off
    paint.setAntiAlias(false);
    paint.setColor(Color.RED);
    canvas.drawRect(100, 5, 200, 30, paint);
    // draw the rotated text
    canvas.rotate(-45);
    paint.setStyle(Paint.Style.FILL);
    canvas.drawText("Graphics Rotation", 40, 180, paint);
   //undo the rotate
    canvas.restore();
```

output



Animation in android

Animation is the process of adding a motion effect to any view, image, or text. With the help of an animation, you can add motion or can change the shape of a specific view. Animation in Android is generally used to give your UI a rich look and feel. The animations are basically of three types as follows:

1. Property Animation
    - Creates an animation by modifying an object's property values over a set period with an `Animator`.
    - Introduced in Android 3.0 (API 11).
    - Works on **object properties** (like alpha, rotation, scaleX, translationX).
    - More powerful and flexible.
2. View Animation
    - **Tween Animation**: Move, Rotate, Scale, Fade.
    - **Frame Animation**: Sequence of images displayed like a flipbook.

Property Animation

Property Animation is one of the robust frameworks which allows animating almost everything.

This is one of the powerful and flexible animations which was introduced in Android 3.0.

Property animation can be used to add any animation in the CheckBox, Radio Buttons, and widgets other than any view.

An animation defined in XML that modifies properties of the target object, such as background color or alpha value, over a set amount of time.

file location:

`res/animator/`*`filename`*`.xml`

The filename is used as the resource ID.

compiled resource datatype:

Resource pointer to a `ValueAnimator`, `ObjectAnimator`, or `AnimatorSet`

resource reference:

In Java-based or Kotlin code: `R.animator.`*`filename`*

In XML: `@[`*`package`*`:]animator/`*`filename`*

```
<set
  android:ordering=["together" | "sequentially"]>

    <objectAnimator
        android:propertyName="string"
        android:duration="int"
        android:valueFrom="float | int | color"
        android:valueTo="float | int | color"
        android:startOffset="int"
        android:repeatCount="int"
        android:repeatMode=["restart" | "reverse"]
        android:valueType=["intType" | "floatType"]/>

    <animator
        android:duration="int"
        android:valueFrom="float | int | color"
        android:valueTo="float | int | color"
        android:startOffset="int"
        android:repeatCount="int"
        android:repeatMode=["restart" | "reverse"]
        android:valueType=["intType" | "floatType"]/>

    <set>
        ...
    </
```

Elements:

`<set>`

A container that holds other animation elements (`<objectAnimator>`, `<valueAnimator>`, or other `<set>` elements). Represents an `AnimatorSet`.

You can specify nested `<set>` tags to further group animations. Each `<set>` can define its own `ordering` attribute.

Attributes:

`android:ordering`

Keyword. Specifies the play ordering of animations in this set.

`sequentially`:Play animations in this set sequentially.
`together` (default):Play animations in this set at the same time.

View Animation

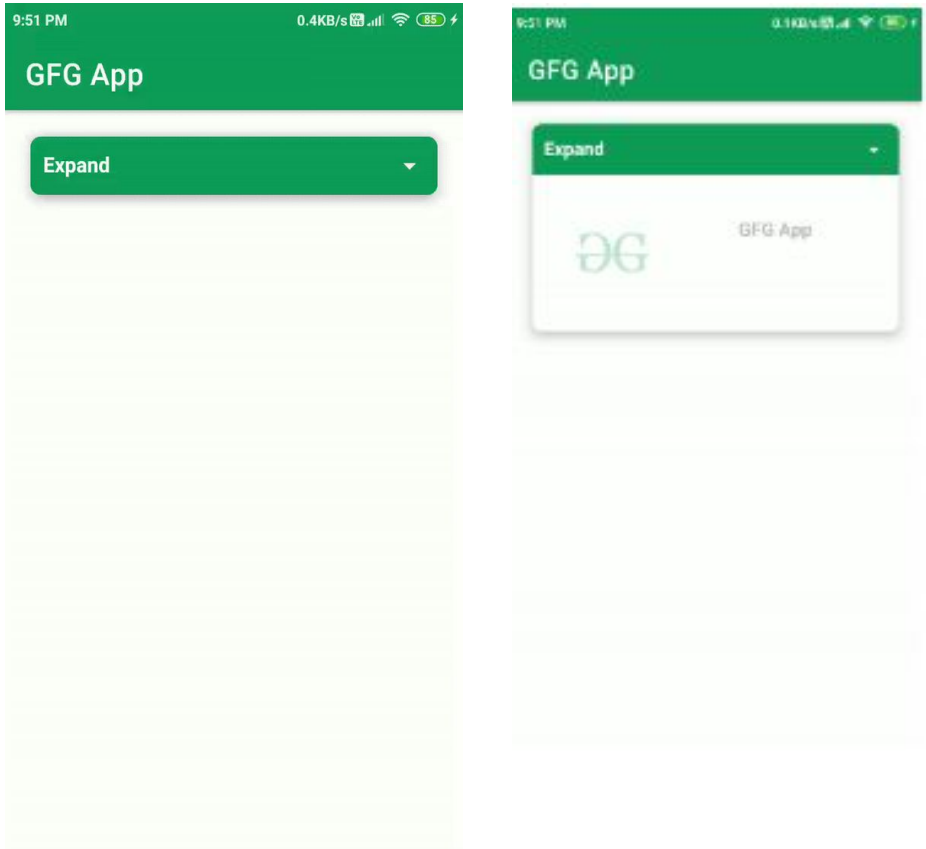
View Animation can be used to add animation to a specific view to perform frame by frame and tweened animation on views.

Tweened animation calculates animation information such as size, rotation, start point, and endpoint. These animations are slower and less flexible.

An example of View animation can be used if we want to expand a specific layout in that place we can use View Animation.

The example of View Animation can be seen in Expandable RecyclerView.

View animation



Frame by Frame animation

- Created with a sequence of different images, played in order, like a roll of film.
- Pieces

➢ The frames in the /res/drawable directory

➢ Animation resource XML file in which the frames are specified as <item> under <animation-list>

➢ Code that loads the Animation resource XML file

Animation XML Resource

- The XML file consists of an <animation-list> element as the root node and a series of child <item> nodes that each define a frame

```
<?xml version="1.0" encoding="UTF-8"?>
<animation-list xmlns:android="http://schemas.android.com/apk/res/android" id="selected"
android:oneshot="false"> <item android:drawable="@drawable/ball1" android:duration="50" />
        <item android:drawable="@drawable/ball2" android:duration="50" />
        <item android:drawable="@drawable/ball3" android:duration="50" />
        <item android:drawable="@drawable/ball4" android:duration="50" />
        <item android:drawable="@drawable/ball5" android:duration="50" />
        <item android:drawable="@drawable/ball6" android:duration="50" />
</animation-list>
```

Code That Starts the Animation

```
public void onCreate(Bundle icicle) {
        super.onCreate(icicle);
       setContentView(R.layout.framebyframe);

// Load the ImageView that will host the animation and // set its background to the AnimationDrawable XML resource.
        ImageView img = (ImageView) findViewById(R.id.my_image_view);
        img.setBackgroundResource(R.anim.simple_animation);

// Get the background, which has been compiled to an AnimationDrawable object.
       frameAnimation = (AnimationDrawable) img.getBackground();
}

public boolean onTouchEvent(MotionEvent event) {
 if (event.getAction() == MotionEvent.ACTION_DOWN) {
       if (animationStarted){
              frameAnimation.stop();
              animationStarted = false;
        } else{
              frameAnimation.start();
              animationStarted = true; }
 return true; }
 return super.onTouchEvent(event);}
```

Tween Animation

Built-in Tween Animation

Schemes:

- TransitionAnimation
  > Position change
- RotateAnimation
  > Rotation
- ScaleAnimation
  > Scaling
- AlphaAnimation
  > Transparancy

Steps of Tween Animation

- Create XML animation resource file in /res/anim/ directory
- Load the XML animation resource file into Animation object
- Start the animation

Create Animation XML Resource

- The XML file is located under /res/anim/ directory
- Example below is for Rotate animation

```
<?xml version="1.0" encoding="UTF-8"?>
 <rotate xmlns:android="http://schemas.android.com/apk/res/android"
android:interpolator="@android:anim/accelerate_decelerate_interpolator"
        android:fromDegrees="0"
        android:toDegrees="270"
        android:toYScale="0.0"
        android:pivotX="50%"
         android:pivotY="50%"
         android:startOffset="700"
        android:duration="700" />
```

Create Animation XML Resource

- **Sequential animation is done with <set>**
- **Example below - Translate animation followed by Rotate animation**

```
<set xmlns:android="http://schemas.android.com/apk/res/android"
        android:shareInterpolator="false">
        <translate
        android:fromXDelta="0"
        android:toXDelta="100"
         android:fromYDelta="0"
        android:toYDelta="150"
         android:duration="1000" />

        <rotate android:interpolator="@android:anim/accelerate_decelerate_interpolator"
        android:fromDegrees="0"
        android:toDegrees="270"
        android:toYScale="0.0"
        android:pivotX="50%"
        android:pivotY="50%"
         android:startOffset="700"
        android:duration="700" />
</set>
```

Loading Animation Resource & Staring Animation

```
// Load Animation object from a resource Animation
 rotate = AnimationUtils.loadAnimation(this, R.anim.rotate);


// Select the animation target and start the animation
 findViewById(R.id.myimage).startAnimation(rotate);
```

Drawable Animation

Drawable Animation is used if you want to animate one image over another.

The simple way to understand is to animate drawable is to load the series of drawable one after another to create an animation.

A simple example of drawable animation can be seen in many apps Splash screen on apps logo animation.

Important methods of animation

startAnimation(): This method will start the animation.
clearAnimation(): This method will clear the animation running on a specific view.

What is Interpolator

- An interpolator defines the rate of change of an animation.
- This allows the basic animation effects (alpha, scale, translate, rotate) to be accelerated, decelerated, repeated, etc.
- You can set an interpolator either in XML animation resource file or programmatically

Interpolators

Built-in Android Interpolators

- Built-in Android Interpolators
- android.R.anim.accelerate_interpolator
- android.R.anim.decelerate_interpolator
- android.R.anim.accelerate_decelerate_interpolator
- android.R.anim.anticipate_interpolator
- android.R.anim.overshoot_interpolator

Example of setting interpolator programmatically

- mAnimation.setInterpolator(AnimationUtils.loadInterpolator(this, android.R.anim.accelerate_interpolator));